



TCC6131 Cloud Computing (July/August 2420)

Project (40%)

Report Submission Due Date: 20th September 2024

Presentation: 23rd September 2024 – 27th September 2024

Activity Overview and Objectives

This activity is designed to help students strengthen the understanding of building a scalable solution on AWS. The proposed solution focuses on an approach to a real-life scenario where customers can start with building the simple, easy to maintain solutions that evolve into a complex, highly available, and scalable architecture. This activity is to be performed in multiple phases.

Upon completion of the activity, students will be able to:

- ✚ Apply some architectural design principles obtained in cloud computing courses
- ✚ Deploy a monolithic PHP, MySQL based application that runs on an Amazon Elastic Compute Cloud (Amazon EC2) instance
- ✚ Refactor the application into multiple tiers
- ✚ Create an RDS instance that the PHP application can query
- ✚ Import and export the MySQL Database Dump
- ✚ Create decoupled architecture using Elastic Load Balancer (ELB)
- ✚ Add autoscaling service to make scalable architectures

General Information

The purpose of this project is to design and develop an online management system. The intent behind this activity is to provide additional hands-on experience to learners outside of pre-defined labs. Only high-level instructions are provided for learners to build a solution without a guide.

You need to enrol in AWS Academy Learner Lab created by your lab instructor. Make sure that there are more than \$20 credits remaining before you begin.

Ideally, you will be working in a group of 4 students. In a special circumstance, you may be allowed by the subject coordinator to attempt the assignment individually, but you will lose marks allocated for effective teamwork.

The project will be marked as a group and any conflict within the group needs to be dealt with via consultation of your lecturer one week before the submission date. If you fail to deal with the conflict on time, the entire group will be responsible for the submission.

Process:

Step 1: Project Release and Group Formation

- ▶ Group formation should be based on the same lab session (maximum 4 members).

Step 2: Rubric

- ▶ Please refer to the project rubric on eBwise under project module for scoring guidance.

Step 3: Submission

- ▶ Please submit your final report on eBwise.

Step 4: Presentation

- ▶ You are required to perform your presentation with your lab instructor.

Prepare A Report Containing the Following Items:

❖ Front Cover

❖ Group Project Declaration Form

❖ Table of Content

❖ Acknowledgement

❖ Phase 1:

- A brief description of your system. Make sure you provide the IP to the website.
- Layout diagram of the network
- Implementation steps:
 - A complete web application running on EC2

❖ Phase 2:

- Layout diagram of the revised network
- Implementation steps:
 - A front-end web application running on the original EC2
 - Creation of a new single-AZ RDS MySQL instance for back-end

❖ Phase 3:

- Layout diagram of the revised network
- Implementation steps:
 - ELB created on a new AZ
 - ELB DNS to access the system

❖ Phase 4:

- Implementation steps of Auto-Scaling setup:
 - EC2
 - Number of instances
 - Scaling policies
- Layout diagram of the revised network

❖ Reflection

❖ References (if any)

❖ Project Rubrics

Problem Statement

An ABC company wants to build an e-commerce inventory management system that allows its employees to monitor and update its inventory. Currently the employees are stocking the inventory as the store is about to open. The leadership team expects business to grow a few months after the store is fully operational. They want to ensure that hundreds of employees can access and update the inventory without worrying about any performance issues. The company has a limited budget to start with and is ready to invest more as business grows.

The company hired your team to present a solution that meets this requirement at minimum cost. To achieve this, your team will build it in phases, starting with single monolithic architecture to a distributed, highly available and scalable system.

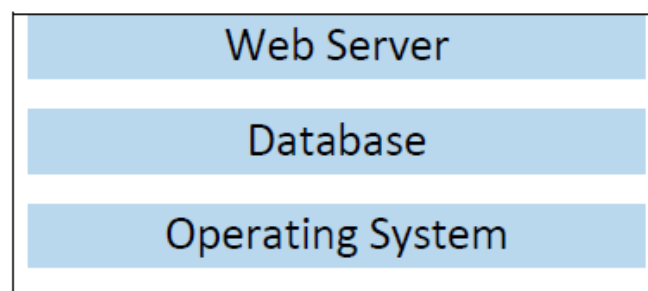
Approach

Modern-day applications grow to meet various demands, so it can be built in phases. When the application is built on cloud it is flexible and has the ability to develop over time. Your team will also build this application in phases. But if you prefer to take a different approach, efforts can vary.

Phase 1 – Build a Simple System (10 marks)

To begin with, you will build a simple web application (app) running on single Amazon Elastic Compute Cloud (Amazon EC2) instance, serving a basic, interactive website with asset information. **You can create your own (or reuse template from other sources).**

This may not follow all the best practices, but it provides you a functional website based on monolithic architecture, which we will be transformed into a distributed one in a future phase.



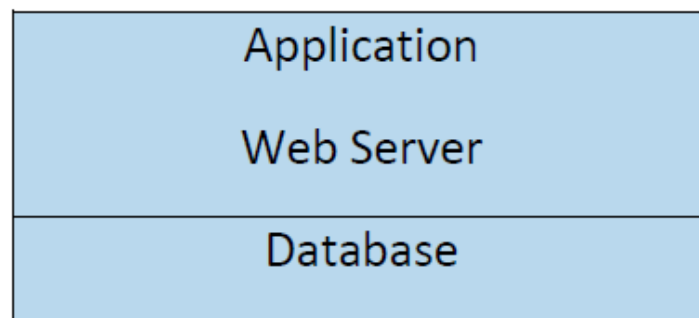
Advantages: Quick to build, fewer components to maintain.

Disadvantages: Monolithic architectures are less flexible, and do not scale easily.

Phase 2 – Break the Monolith (5 marks)

Now that you have a functional app, you can think about applying the AWS Well Architected Framework. Note that there is no best architecture design, because it is always driven by various factors depending upon use case.

As a first step towards breaking the monolith, separate the database from web application server, so the web app and databases run on separate servers. This is a two-tier architecture.



Advantages:

- Separating gives ability to manage and scale them separately.
- Reduces the risk of having everything on one EC2 instance (avoiding single point of failure).

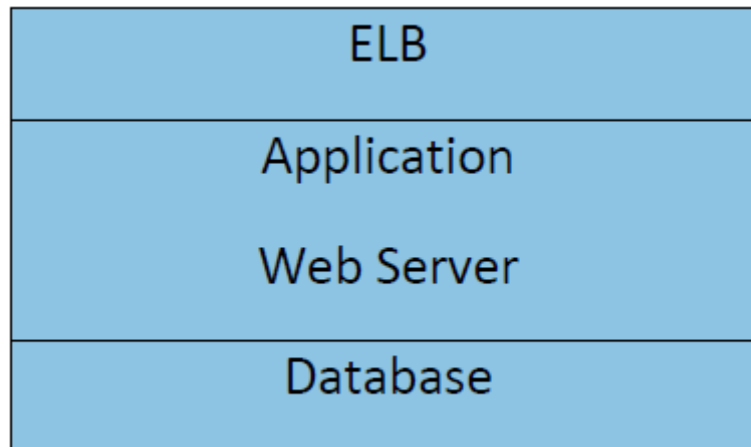
Disadvantages:

- Adding more components increases cost.
- Code changes required to establish connectivity between components after splitting it up.

Phase 3 – Decoupling (5 marks)

Now that you separated the web server from the databases, both of them can perform independently without competing for resources on the same server. Both layers can be scaled independently depending on the performance needs.

Next step would be to decouple the web server from the users by adding Elastic Load Balancer (ELB) and transform this into a three-tier architecture.



Phase 4 – Scaling (5 marks)

Now you have a working decoupled multi-layered architecture of the application where you separated various layers. The systems are serving a few requests per second with no visible performance issues. The team needs to prepare for an increase of visitors to the website to maintain performance.

Introduce auto-scaling for web app layer, which will horizontally scale the web app (by adding or removing EC2 instances) in response to the demand on the website.

